1. Aim

To write a Java program that displays all possible letter combinations for a given string of digits using the phone keypad mapping.

2. Description

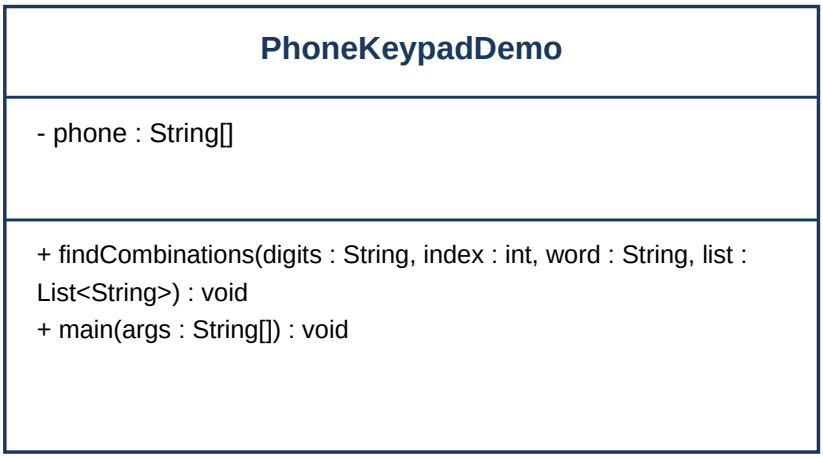
The program maps digits 2 through 9 to their corresponding letters on a phone keypad. The user enters a sequence of digits. The findCombinations() method uses recursion to build each possible word and stores the completed combinations in a List. The program then prints the combinations and their total count. If the input contains anything other than digits 2 to 9, it displays the validation message.

3. Software Requirements

Component	Requirement
Hardware	Computer or laptop
Operating system	Windows, Linux, or macOS
Software	JDK (Java Development Kit) 8 or later
Editor / IDE	Notepad, VS Code, Eclipse, IntelliJ IDEA, or any Java IDE
File name	PhoneKeypadDemo.java

4. UML Diagram

The UML class diagram shows the PhoneKeypadDemo class, its phone mapping, and its methods.



5. Java Code

The code below is reproduced as provided, without changes.

```
package recordprograms;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class PhoneKeypadDemo
{
    static String[] phone = {
        "", "", "abc", "def", "ghi",
        "jkl", "mno", "pqrs", "tuv", "wxyz"
    };

    public static void findCombinations(String digits, int index,
                                        String word, List<String> list)
    {
        if (index == digits.length())
        {
            list.add(word);
            return;
        }

        int num = digits.charAt(index) - '0';
        String letters = phone[num];

        for (int i = 0; i < letters.length(); i++)
        {
            findCombinations(digits, index + 1,
                    word + letters.charAt(i), list);
        }
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter digits: ");
        String digits = sc.nextLine().trim();

        List<String> result = new ArrayList<>();

        if (!digits.matches("[2-9]+"))
        {
            System.out.println("Enter digits from 2 to 9 only.");
        }
        else
        {
            findCombinations(digits, 0, "", result);

            System.out.println("Combinations: " + result);
            System.out.println("Total combinations: " + result.size());
        }

        sc.close();
    }
}
```

6. Sample Output

Example input: 23

```
Enter digits: 23
Combinations: [ad, ae, af, bd, be, bf, cd, ce, cf]
Total combinations: 9
```

Note: This is a sample terminal-style output illustration, not a screenshot captured from an actual Java run. The sample input is 23.

7. Result

The program generates and displays all possible letter combinations for valid digits from 2 to 9 using recursion, and prints the total number of combinations.

How to compile and run

```
javac -d . PhoneKeypadDemo.java
java recordprograms.PhoneKeypadDemo
```